



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش چهارم

توصیف رفتاری (پشته‌ی LIFO)

خلاصه

در این آزمایش یک پشته‌ی LIFO با عمق ۸ کلمه و پهنای ۴ بیت با توصیف رفتاری (behavioral) در Verilog پیاده‌سازی شده است. برخلاف آزمایش قبل که همه‌چیز با assign نوشته می‌شد، اینجا کل مازول با بلوک‌های always توصیف می‌شود: یک بلوک ترتیبی همگام با لبه‌ی بالارونده‌ی کلاک برای حافظه و اشاره‌گر پشته، و یک بلوک ترکیبی (*@always برای خروجی داده.

صحت‌سنجی با Icarus Verilog (vvp/iverilog) و به کمک یک مدل مرجع مستقل انجام شده است؛ مدل مرجع در تست‌بنچ به‌صورت جداگانه پیاده شده و خروجی طراحی در هر لبه‌ی کلاک با آن مقایسه می‌شود.

مشخصات واسط

ورودی‌ها و خروجی‌های خواسته‌شده در صورت آزمایش به این شکل پیاده شده‌اند:

سیگنال	جهت	پهنای	توضیح
Clk	ورودی	۱	کلاک؛ همه‌ی تغییرات حالت روی لبه‌ی بالارونده
RstN	ورودی	۱	ریست ناهمگام فعال‌پایین؛ پشته را خالی می‌کند
Data_In	ورودی	۴	داده‌ی ورودی برای Push
Push	ورودی	۱	فرمان نوشتن روی سر پشته
Pop	ورودی	۱	فرمان برداشتن از سر پشته
Data_Out	خروجی	۴	کلمه‌ی روی سر پشته (خواندن ترکیبی)
Full	خروجی	۱	Full = ۱ یعنی پشته پر است
Empty	خروجی	۱	Empty = ۱ یعنی پشته خالی است

نام سیگنال ریست در صورت آزمایش RstN است؛ پسوند N قرارداد رایج برای فعال‌پایین بودن است، بنابراین ریست به‌صورت ناهمگام و فعال‌پایین (negedge RstN) پیاده شده است.

در صورت آزمایش نوشته شده Empty=0 indicates that the stack is empty که با سطر بالایی آن Full=1 indicates that the stack is full (ناسازگار است؛ اگر ۰ Empty به معنی خالی بودن باشد، آنگاه پرچم عملاً یک سیگنال Not-Empty است و نامش گمراه‌کننده می‌شود. در این پیاده‌سازی قرارداد متعارف و متقارن با Full انتخاب شده است، یعنی ۱ Empty به معنی خالی بودن پشته. اگر مقصود صورت آزمایش دقیقاً همان متن باشد، تنها کافی است خروجی معکوس شود:

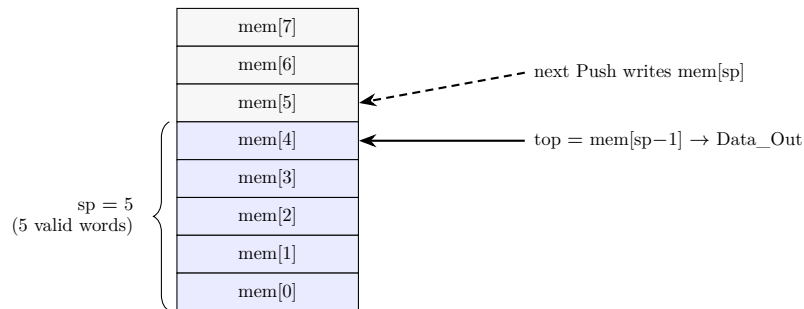
```
assign Empty = ~(sp == 0);
```

ساختار داخلی

پشته با یک آرایه‌ی حافظه و یک اشاره‌گر توصیف می‌شود. اشاره‌گر sp برابر تعداد کلمات معتبر داخل پشته است، نه اندیس خانه‌ی بالایی؛ این انتخاب باعث می‌شود پرچم‌ها بسیار ساده شوند:

$$\begin{aligned} \text{Empty} &= (\text{sp} = 0) \\ \text{Full} &= (\text{sp} = \text{DEPTH}) \\ \text{Data_Out} &= \begin{cases} 0 & \text{sp} = 0 \\ \text{mem}[\text{sp} - 1] & \text{در غیر این صورت} \end{cases} \end{aligned}$$

چون sp باید بتواند هر دو مقدار ۰ و DEPTH را نگه دارد، پهنای آن $1 + \lceil \log_2 \text{DEPTH} \rceil$ بیت است (برای عمق ۸ یعنی ۴ بیت). اگر اشتباه ۳ بیت گرفته می‌شد، مقدار ۸ به ۰ سرریز می‌کرد و پشته‌ی پر با پشته‌ی خالی اشتباه گرفته می‌شد.



داوری فرمان‌ها

چهار حالت مرزی وجود دارد که باید صریحاً مشخص شوند، وگرنه مدار در شرایط لبه‌ای رفتار نامعین پیدا می‌کند:

وضعیت	رفتار	دلیل
Push و پشته پر	نادیده گرفته می‌شود	جلوگیری از overflow؛ داده‌ی قدیمی خراب نمی‌شود
Pop و پشته خالی	نادیده گرفته می‌شود	جلوگیری از underflow؛ sp منفی نمی‌شود
Push و Pop همزمان	هیچ کاری انجام نمی‌شود	دو فرمان مستقل و متناقض؛ امن‌ترین تفسیر
هیچ‌کدام فعال نیست	حالت حفظ می‌شود	

این داوری در دو سیگنال کمکی خلاصه شده است تا بدنه‌ی بلوک `always` تمیز بماند:

```
wire do_push = Push & ~Pop & ~Full;
wire do_pop  = Pop  & ~Push & ~Empty;
```

توجه کنید که برای `Push` و `Pop` همزمان می‌شد رفتار `replace top` (جایگزینی سر پشته) را هم انتخاب کرد؛ اما صورت آزمایش این دو را دو فرمان مستقل تعریف کرده و چیزی درباره‌ی ترکیبشان نگفته است، پس تفسیر محافظه‌کارانه‌ی بدون تغییر انتخاب شده و در تست‌بنچ هم صریحاً آزموده می‌شود.

کد طراحی

ماژول کاملاً رفتاری است: حافظه و اشاره‌گر در یک `@(posedge Clk or negedge RstN)` و `always` خروجی داده در یک `@(*)` توصیف شده‌اند. پارامترهای `WIDTH` و `DEPTH` پیش‌فرض همان مقادیر خواسته‌شده در صورت آزمایش هستند.

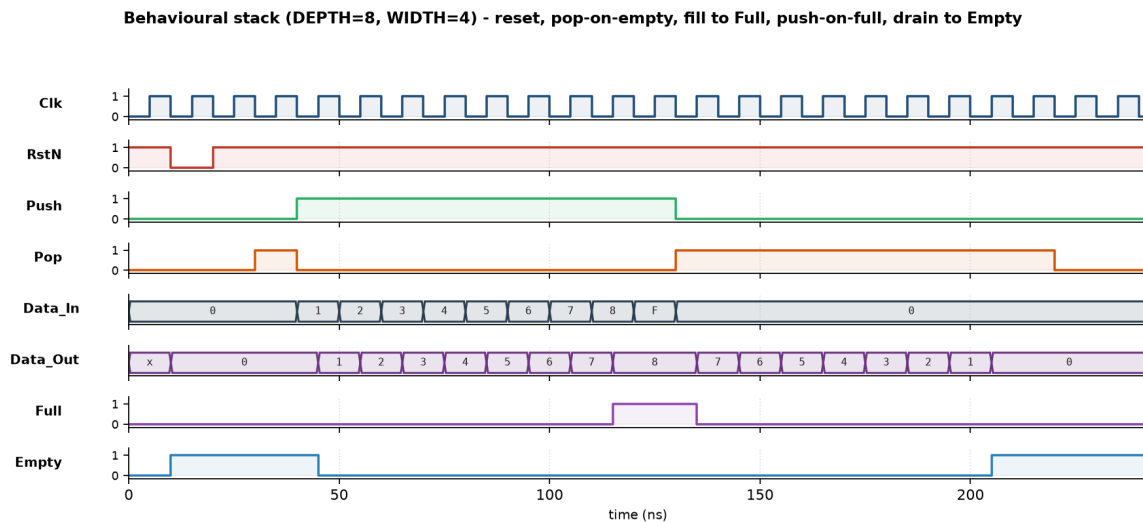
```

1 module stack #(
2     parameter integer WIDTH = 4,
3     parameter integer DEPTH = 8
4 ) (
5     input wire          Clk,
6     input wire          RstN,
7     input wire [WIDTH-1:0] Data_In,
8     input wire          Push,
9     input wire          Pop,
10    output reg [WIDTH-1:0] Data_Out,
11    output wire          Full,
12    output wire          Empty
13 );
14 localparam integer PTR_W = $clog2(DEPTH) + 1;
15
16 reg [WIDTH-1:0] mem [0:DEPTH-1];
17 reg [PTR_W-1:0] sp;
18
19 integer i;
20
21 assign Empty = (sp == 0);
22 assign Full  = (sp == DEPTH[PTR_W-1:0]);
23
24 wire do_push = Push & ~Pop & ~Full;
25 wire do_pop  = Pop  & ~Push & ~Empty;
26
27 always @(posedge Clk or negedge RstN) begin
28     if (!RstN) begin
29         sp <= {PTR_W{1'b0}};
30         for (i = 0; i < DEPTH; i = i + 1)
31             mem[i] <= {WIDTH{1'b0}};
32     end else begin
33         if (do_push) begin
34             mem[sp] <= Data_In;
35             sp <= sp + 1'b1;
36         end else if (do_pop) begin
37             sp <= sp - 1'b1;
38         end
39     end
40 end
41
42 always @(*) begin
43     if (sp == 0)
44         Data_Out = {WIDTH{1'b0}};
45     else
46         Data_Out = mem[sp - 1'b1];
47 end
48 endmodule

```

شکل موج

سناریوی شکل موج طوری چیده شده که همه‌ی رفتارهای مرزی در یک تصویر دیده شوند: ابتدا ریست ناهم‌گام (پشته خالی و ۱ = Empty)، سپس یک Pop روی پشته‌ی خالی که نادیده گرفته می‌شود، سپس هشت Push پیاپی با مقادیر ۱ تا ۸، سپس یک Push با مقدار F روی پشته‌ی پر که نباید اثری بگذارد، و در پایان هشت Pop که داده‌ها را دقیقاً به ترتیب معکوس باز می‌گردانند.



چند نکته در شکل بالا قابل مشاهده است. پرچم Full دقیقاً روی هشتمین Push بالا می‌رود و نه زودتر؛ در بازه‌ای که Full فعال است و Data_In برابر F می‌شود، مقدار Data_Out روی ۸ ثابت می‌ماند که یعنی سرریز واقعاً بلوکه شده است. در بخش تخلیه، Data_Out دنباله‌ی ۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸ را طی می‌کند که همان ترتیب LIFO است، و در انتها Empty دوباره فعال شده و Data_Out به ۰ برمی‌گردد.

تست‌بنج اصلی

تست‌بنج به جای مقایسه با مقادیر ثابت دستی، یک مدل مرجع مستقل نگه می‌دارد (آرایه‌ی ref_mem و شمارنده‌ی ref_sp) و بعد از هر لبه‌ی کلاک، هر سه خروجی Data_Out، Full و Empty را با مدل مقایسه می‌کند. تحریک ورودی‌ها روی لبه‌ی پایین‌رونده اعمال می‌شود تا شرایط setup/hold حول لبه‌ی بالارونده هیچ‌وقت مبهم نباشد.

سناریوهای پوشش داده‌شده به ترتیب عبارت‌اند از: ریست ناهم‌گام، Pop روی پشته‌ی خالی، پرکردن کامل تا Full، Push روی پشته‌ی پر، تخلیه‌ی کامل و بررسی ترتیب LIFO، سیکل‌های بیکار، Push و Pop همزمان در سه عمق مختلف (خالی، میانه، پر)، ریست از حالت غیرخالی، الگوی درهم‌آمیخته‌ی push/pop، و در پایان ۲۰۰۰ فرمان تصادفی که مستقیماً با مدل مرجع مقایسه می‌شوند.

```

1 `timescale 1ns/1ps
2
3 module tb_stack;
4     localparam integer WIDTH = 4;
5     localparam integer DEPTH = 8;
6
7     reg                Clk, RstN, Push, Pop;
8     reg [WIDTH-1:0]    Data_In;
9     wire [WIDTH-1:0]   Data_Out;
10    wire               Full, Empty;
11
12    stack #(.WIDTH(WIDTH), .DEPTH(DEPTH)) dut (
13        .Clk(Clk), .RstN(RstN), .Data_In(Data_In),
14        .Push(Push), .Pop(Pop),
15        .Data_Out(Data_Out), .Full(Full), .Empty(Empty)
16    );
17
18    reg [WIDTH-1:0] ref_mem [0:DEPTH-1];
19    integer         ref_sp;
20
21    integer pass, fail;
22    integer i, k, iter;
23    reg [WIDTH-1:0] exp_dout;
24    reg             exp_full, exp_empty;
25
26    initial Clk = 1'b0;
27    always #5 Clk = ~Clk;
28
29    task automatic ref_reset;
30        begin
31            ref_sp = 0;
32            for (i = 0; i < DEPTH; i = i + 1) ref_mem[i] = {WIDTH{1'b0}};
33        end
34    endtask
35
36    task automatic step;
37        input          p_push;
38        input          p_pop;
39        input [WIDTH-1:0] p_din;
40        input [255:0]    tag;
41        begin
42            @(negedge Clk);
43            Push = p_push;
44            Pop = p_pop;
45            Data_In = p_din;
46
47            @(posedge Clk);
48            if (p_push && !p_pop && ref_sp < DEPTH) begin
49                ref_mem[ref_sp] = p_din;
50                ref_sp = ref_sp + 1;
51            end else if (p_pop && !p_push && ref_sp > 0) begin
52                ref_sp = ref_sp - 1;
53            end
54
55            #1;
56            check(tag);
57        end
58    endtask
59
60    task automatic check;
61        input [255:0] tag;
62        begin
63            exp_empty = (ref_sp == 0);
64            exp_full = (ref_sp == DEPTH);
65            exp_dout = (ref_sp == 0) ? {WIDTH{1'b0}} : ref_mem[ref_sp-1];
66
67            if (Data_Out === exp_dout && Full === exp_full && Empty === exp_empty) begin
68                pass = pass + 1;
69            end else begin

```

```

70         $display("FAIL [%0s] sp=%0d got D=%h F=%b E=%b exp D=%h F=%b E=%b",
71                 tag, ref_sp, Data_Out, Full, Empty,
72                 exp_dout, exp_full, exp_empty);
73         fail = fail + 1;
74     end
75 end
76 endtask
77
78 task automatic do_reset;
79     begin
80         @(negedge Clk);
81         Push = 0; Pop = 0; Data_In = 0;
82         RstN = 1'b0;
83         @(negedge Clk);
84         RstN = 1'b1;
85         ref_reset;
86         #1;
87         check("reset");
88     end
89 endtask
90
91 initial begin
92     pass = 0; fail = 0;
93     RstN = 1'b1; Push = 0; Pop = 0; Data_In = 0;
94     ref_reset;
95
96     do_reset;
97
98     step(1'b0, 1'b1, 4'h0, "pop_when_empty_1");
99     step(1'b0, 1'b1, 4'h0, "pop_when_empty_2");
100
101     for (k = 0; k < DEPTH; k = k + 1)
102         step(1'b1, 1'b0, k[WIDTH-1:0] + 4'h1, "fill");
103
104     step(1'b1, 1'b0, 4'hF, "push_when_full_1");
105     step(1'b1, 1'b0, 4'hE, "push_when_full_2");
106
107     for (k = 0; k < DEPTH; k = k + 1)
108         step(1'b0, 1'b1, 4'h0, "drain");
109
110     step(1'b0, 1'b0, 4'h5, "idle_empty");
111
112     step(1'b1, 1'b1, 4'hA, "both_at_empty");
113     step(1'b1, 1'b0, 4'h3, "seed_mid");
114     step(1'b1, 1'b0, 4'h7, "seed_mid");
115     step(1'b1, 1'b1, 4'hC, "both_at_mid");
116     step(1'b0, 1'b0, 4'h0, "idle_mid");
117     for (k = 0; k < DEPTH; k = k + 1)
118         step(1'b1, 1'b0, 4'h9, "refill");
119     step(1'b1, 1'b1, 4'hB, "both_at_full");
120
121     do_reset;
122
123     step(1'b1, 1'b0, 4'h1, "int_push1");
124     step(1'b1, 1'b0, 4'h2, "int_push2");
125     step(1'b0, 1'b1, 4'h0, "int_pop2");
126     step(1'b1, 1'b0, 4'h3, "int_push3");
127     step(1'b1, 1'b0, 4'h4, "int_push4");
128     step(1'b0, 1'b1, 4'h0, "int_pop4");
129     step(1'b0, 1'b1, 4'h0, "int_pop3");
130     step(1'b0, 1'b1, 4'h0, "int_pop1");
131
132     do_reset;
133     for (iter = 0; iter < 2000; iter = iter + 1)
134         step($random, $random, $random, "random");
135
136     $display("tb_stack: Passed: %0d, Failed: %0d", pass, fail);
137     if (fail != 0) $fatal(1);
138     $finish;
139 end
140
141 initial begin
142     #500000;
143     $display("FAIL: timeout");
144     $fatal(1);
145 end
146 endmodule

```

تست‌بنچ پارامتری

علاوه بر تست اصلی، یک تست‌بنچ دوم همان مازول را با سه هندسه‌ی متفاوت می‌سازد: (WIDTH, DEPTH) برابر (۴, ۸) یعنی همان چیزی که صورت آزمایش خواسته، و همچنین (۸, ۴) و (۲, ۱۶). هدف این است که مطمئن شویم عمق و پهنا واقعا پارامتری‌اند و جایی در کد به‌صورت ثابت فرض نشده‌اند؛ به‌ویژه محاسبه‌ی پهنای اشاره‌گر که در صورت اشتباه، دقیقا در حالت Full خودش را نشان می‌دهد.

```

1 `timescale 1ns/1ps
2
3 module tb_stack_param;
4
5     integer pass, fail;
6
7     `define MAKE_CASE(NAME, W, D) \
8     reg      Clk`NAME, RstN`NAME, Push`NAME, Pop`NAME; \
9     reg [W-1:0] Din`NAME; \
10    wire [W-1:0] Dout`NAME; \
11    wire      Full`NAME, Empty`NAME; \
12    stack #(.WIDTH(W), .DEPTH(D)) dut`NAME ( \
13        .Clk(Clk`NAME), .RstN(RstN`NAME), .Data_In(Din`NAME), \
14        .Push(Push`NAME), .Pop(Pop`NAME), .Data_Out(Dout`NAME), \
15        .Full(Full`NAME), .Empty(Empty`NAME)); \
16    initial Clk`NAME = 1'b0; \
17    always #5 Clk`NAME = ~Clk`NAME;
18
19    `MAKE_CASE(A, 4, 8)
20    `MAKE_CASE(B, 8, 4)
21    `MAKE_CASE(C, 2, 16)
22
23    task automatic report;
24        input [255:0] tag;
25        input        ok;
26        begin
27            if (ok) pass = pass + 1;
28            else begin
29                $display("FAIL [%0s]", tag);
30                fail = fail + 1;
31            end
32        end
33    endtask
34
35    integer k;
36
37    task automatic run_A;
38        begin
39            @(negedge ClkA); RstNA = 0; PushA = 0; PopA = 0; DinA = 0;
40            @(negedge ClkA); RstNA = 1; #1;
41            report("A/empty_after_reset", EmptyA === 1'b1 && FullA === 1'b0);
42
43            for (k = 0; k < 8; k = k + 1) begin
44                @(negedge ClkA); PushA = 1; PopA = 0; DinA = k[3:0];
45                @(posedge ClkA); #1;
46                report("A/fill_flags",
47                    EmptyA === 1'b0 && FullA === ((k == 7) ? 1'b1 : 1'b0)
48                    && DoutA === k[3:0]);
49            end
50            @(negedge ClkA); PushA = 1; DinA = 4'hF;
51            @(posedge ClkA); #1;
52            report("A/overflow_ignored", FullA === 1'b1 && DoutA === 4'd7);
53
54            for (k = 7; k >= 0; k = k - 1) begin
55                @(negedge ClkA); PushA = 0; PopA = 1;
56                @(posedge ClkA); #1;
57                report("A/drain_lifo",
58                    (k == 0) ? (EmptyA === 1'b1 && DoutA === 4'h0)
59                    : (DoutA === (k[3:0] - 4'h1) && EmptyA === 1'b0));
60            end
61            @(negedge ClkA); PopA = 1;
62            @(posedge ClkA); #1;
63            report("A/underflow_ignored", EmptyA === 1'b1 && DoutA === 4'h0);
64            @(negedge ClkA); PopA = 0;
65        end
66    endtask
67
68    task automatic run_B;
69        begin
70            @(negedge ClkB); RstNB = 0; PushB = 0; PopB = 0; DinB = 0;
71            @(negedge ClkB); RstNB = 1; #1;
72            report("B/empty_after_reset", EmptyB === 1'b1 && FullB === 1'b0);
73
74            for (k = 0; k < 4; k = k + 1) begin
75                @(negedge ClkB); PushB = 1; PopB = 0; DinB = 8'hA0 + k[7:0];
76                @(posedge ClkB); #1;

```

```

77         report("B/fill_flags",
78             FullB == ((k == 3) ? 1'b1 : 1'b0)
79             && DoutB == (8'hA0 + k[7:0]));
80     end
81     @(negedge ClkB); PushB = 1; DinB = 8'hFF;
82     @(posedge ClkB); #1;
83     report("B/overflow_ignored", FullB == 1'b1 && DoutB == 8'hA3);
84
85     for (k = 3; k >= 0; k = k - 1) begin
86         @(negedge ClkB); PushB = 0; PopB = 1;
87         @(posedge ClkB); #1;
88         report("B/drain_lifo",
89             (k == 0) ? (EmptyB == 1'b1
90                 : (DoutB == (8'hA0 + k[7:0] - 8'h1)));
91     end
92     @(negedge ClkB); PopB = 0;
93 end
94 endtask
95
96 task automatic run_C;
97     begin
98         @(negedge ClkC); RstNC = 0; PushC = 0; PopC = 0; DinC = 0;
99         @(negedge ClkC); RstNC = 1; #1;
100        report("C/empty_after_reset", EmptyC == 1'b1 && FullC == 1'b0);
101
102        for (k = 0; k < 16; k = k + 1) begin
103            @(negedge ClkC); PushC = 1; PopC = 0; DinC = k[1:0];
104            @(posedge ClkC); #1;
105            report("C/fill_flags",
106                FullC == ((k == 15) ? 1'b1 : 1'b0) && DoutC == k[1:0]);
107        end
108        @(negedge ClkC); PushC = 1; DinC = 2'b11;
109        @(posedge ClkC); #1;
110        report("C/overflow_ignored", FullC == 1'b1 && DoutC == 2'b11);
111
112        for (k = 15; k >= 0; k = k - 1) begin
113            @(negedge ClkC); PushC = 0; PopC = 1;
114            @(posedge ClkC); #1;
115            report("C/drain_lifo",
116                (k == 0) ? (EmptyC == 1'b1
117                    : (DoutC == (k[1:0] - 2'b01)));
118        end
119        @(negedge ClkC); PopC = 0;
120    end
121 endtask
122
123 initial begin
124     pass = 0; fail = 0;
125     run_A;
126     run_B;
127     run_C;
128     $display("Passed: %0d, Failed: %0d", pass, fail);
129     if (fail != 0) $fatal(1);
130     $finish;
131 end
132 endmodule

```

نتایج تست

```

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_stack ===
tb_stack: Passed: 2046, Failed: 0
PASS: tb_stack

=== tb_stack_param ===
tb_stack_param: Passed: 63, Failed: 0
PASS: tb_stack_param

summary: 2/2 passed, 0 failed

```

جمع‌بندی

تفاوت اصلی این آزمایش با آزمایش قبل در سطح توصیف است. در توصیف جریان داده باید خود معادلات منطقی نوشته می‌شد، اما در توصیف رفتاری کافی است رفتار مدار را بیان کنیم و ساخت حافظه، شمارنده و منطق کنترلی به ابزار سنتز سپرده شود. همین باعث می‌شود پیاده‌سازی پشت‌پای که در سطح گیت کار نسبتاً سنگینی است، در چند خط always خلاصه شود.

نکته‌ی مهم دیگر این است که در توصیف رفتاری، حالت‌های مرزی به‌طور خودکار امن نمی‌شوند؛ سرریز، تهی‌خوانی و فرمان‌های متناقض باید صریحاً در کد مدیریت شوند، وگرنه اشاره‌گر پشت‌پای می‌تواند از محدوده خارج شود. به همین دلیل بخش عمده‌ی تست‌بنچ به همین حالت‌ها اختصاص یافته است.